Modelos de difusión para IA generativa

Guía de configuración del curso

Preparación del entorno, dataset y estructura de ejecución

Semillero GIDSYC | Universidad del Magdalena

Versión del curso | 2026-I

Esta guía define la configuración mínima para ejecutar el curso de manera reproducible. El objetivo no es que el estudiante invierta tiempo armando datasets, sino que llegue al notebook con el entorno, las rutas y los archivos necesarios en estado verificable.

1 Propósito

El curso trabaja con notebooks de Python que estudian modelos de difusión desde la formulación estadística hasta DDPM, condicionamiento, classifier-free guidance, score matching, flow matching y espacios latentes. Para que la secuencia sea estable, el estudiante debe ejecutar los notebooks desde una raíz de proyecto consistente y con los archivos de datos ya preparados.

La configuración recomendada asume que el estudiante dispone de la carpeta del curso, un entorno de Python funcional, los datasets procesados y los checkpoints pequeños incluidos en la distribución. La prioridad es verificar que las rutas y dependencias estén listas antes de iniciar la ejecución de notebooks.

> **Decisión operativa.** Para clase, se recomienda usar los archivos procesados `.npz` incluidos con el curso. Descargar y reprocesar QuickDraw no es una carga inicial necesaria para estudiar la ruta estándar.

2 Raíz del curso

La carpeta que debe abrirse como proyecto es:

```
courses/modelos_difusion_ia_generativa/
```

Desde esa carpeta, la estructura relevante debe ser:

```
notebooks/      Cuadernos del curso
materiales/     Notas técnicas y figuras complementarias
assets/         Branding y figuras usadas por el curso
```

```
data/           Datasets procesados de QuickDraw
models/         Checkpoints usados por notebooks posteriores
src/            Utilidades mínimas importadas por los notebooks
docs/           Documentación de ruta y criterios del curso
```

Los notebooks están escritos para detectar la raíz cuando se ejecutan desde `notebooks/` o desde la carpeta del curso. Aun así, la práctica más limpia es abrir JupyterLab desde `courses/modelos_difusion_ia_generativa/`.

3 Entorno de Python

Se recomienda usar Python 3.10 o superior. El curso fue diseñado para depender de un conjunto pequeño de librerías: `numpy`, `matplotlib` y `torch`. Para ejecutar notebooks localmente también se requiere `jupyterlab` e `ipykernel`.

3.1 Opción con conda

```
conda create -n dl-env python=3.10 -y
conda activate dl-env
python -m pip install --upgrade pip
cd courses/modelos_difusion_ia_generativa
python -m pip install -r requirements.txt
python -m pip install jupyterlab ipykernel
python -m ipykernel install --user \
  --name dl-env \
  --display-name "Python (dl-env)"
```

3.2 Opción con venv

```
cd courses/modelos_difusion_ia_generativa
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python -m pip install jupyterlab ipykernel
python -m ipykernel install --user \
  --name difusion-curso \
  --display-name "Python (difusion-curso)"
```

3.3 PyTorch y GPU

El archivo `requirements.txt` declara `torch>=2.0`. En CPU, esto suele ser suficiente para los notebooks conceptuales y visuales. Para entrenamiento y muestreo se recomienda GPU. La instalación exacta de PyTorch con CUDA depende del sistema operativo, versión de CUDA soportada y controlador instalado; por eso la referencia oficial debe ser el selector de instalación de PyTorch:

https://pytorch.org/get-started/

La verificación mínima dentro del entorno es:

```
python - <<'PY'
import torch
print("PyTorch:", torch.__version__)
print("CUDA disponible:", torch.cuda.is_available())
if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
PY
```

4 Dataset QuickDraw

El curso usa el dataset QuickDraw de Google Creative Lab, publicado bajo licencia CC BY 4.0. La fuente oficial es:

https://github.com/googlecreativelab/quickdraw-dataset

La versión operativa del curso no usa los archivos originales directamente. Los notebooks esperan archivos `.npz` procesados con imágenes de 28×28 y partición entrenamiento-validación.

4.1 Archivos esperados

Dentro de la raíz del curso deben existir:

```
data/quickdraw/processed/quickdraw_house_50k_28x28.npz
data/quickdraw/processed/quickdraw_5class_10k_each_28x28.npz
data/quickdraw/metadata/quickdraw_house_50k_28x28.json
data/quickdraw/metadata/quickdraw_5class_10k_each_28x28.json
```

El primer archivo contiene la categoría `house` y se usa en los notebooks iniciales. El segundo contiene cinco clases balanceadas: `house`, `cat`, `tree`, `fish` y `apple`; se usa desde el bloque de mezclas y condicionamiento.

4.2 Verificación del dataset

Desde la raíz del curso:

```
python - <<'PY'
from pathlib import Path
import numpy as np

root = Path.cwd()
paths = [
    root / "data/quickdraw/processed/quickdraw_house_50k_28x28.npz",
    root / "data/quickdraw/processed/quickdraw_5class_10k_each_28x28.npz",
]

for path in paths:
    print(path.relative_to(root), "existe:", path.exists())
```

```
    if path.exists():
        data = np.load(path)
        print("  claves:", sorted(data.files))
        print("  x_train:", data["x_train"].shape, data["x_train"].dtype)
        print("  x_val:", data["x_val"].shape, data["x_val"].dtype)
        print("  clases:", data["class_names"].tolist())
PY
```

5 Checkpoints del curso

Los notebooks de entrenamiento pueden crear checkpoints si no existen, pero la distribución estable del curso debe incluirlos para evitar que todos los estudiantes entrenen desde cero.

Los archivos esperados son:

```
models/quickdraw_house_ddpm_tiny_unet.pt
models/quickdraw_5class_conditional_tiny_unet.pt
models/quickdraw_5class_cfg_tiny_unet.pt
```

La verificación mínima es:

```
python - <<'PY'
from pathlib import Path

for path in sorted(Path("models").glob("*.pt")):
    size_mb = path.stat().st_size / 1024**2
    print(f"{path}: {size_mb:.2f} MiB")
PY
```

> **Criterio de ejecución.** Si el objetivo de la sesión es estudiar muestreo, condicionamiento o guidance, conviene usar checkpoints ya entrenados. Si el objetivo es estudiar entrenamiento, puede ejecutarse el entrenamiento completo, preferiblemente con GPU.

6 Ejecución de notebooks

El orden recomendado es estrictamente secuencial:

```
00_distribuciones_de_datos_y_modelado_generativo.ipynb
01_ruido_gaussiano_y_espacios_de_datos.ipynb
02_procesos_de_difusion_directa.ipynb
03_denoising_y_objetivos_de_entrenamiento.ipynb
04_modelo_ddpm_no_condicional_prediccion_ruido.ipynb
05_muestreo_iterativo_y_evaluacion_cualitativa.ipynb
06_mezclas_de_distribuciones_y_estructura_semantica.ipynb
07_modelos_de_difusion_condicionales.ipynb
08_classifier_free_guidance.ipynb
```

```
09_score_matching_e_intuicion_geometrica.ipynb
10_flow_matching_y_transporte_de_probabilidad.ipynb
11_espacios_latentes_y_arquitecturas_generativas_modernas.ipynb
```

Para iniciar JupyterLab:

```
cd courses/modelos_difusion_ia_generativa
jupyter lab
```

En Jupyter, seleccione el kernel creado para el curso. Si se usa conda, el nombre sugerido es `Python (dl-env)`.

7 Verificaciones y problemas frecuentes

Antes de iniciar el curso, revise estos puntos operativos:

- **Dataset no encontrado.** Verifique que Jupyter fue iniciado desde la raíz del curso o desde `notebooks/`. Después confirme que existen los archivos de `data/quickdraw/processed/`. El error más común es abrir Jupyter desde una carpeta superior y ejecutar celdas con una raíz inesperada.
- **Kernel sin PyTorch.** El entorno correcto puede estar instalado, pero Jupyter puede estar usando otro kernel. Revise el nombre del kernel en la esquina superior derecha del notebook. Si no aparece, registre el kernel con:

  ```
  python -m ipykernel install --user \
    --name dl-env \
    --display-name "Python (dl-env)"
  ```

- **GPU no disponible.** Los notebooks detectan GPU con `torch.cuda.is_available()`. Si no hay GPU, varias secciones siguen funcionando en CPU, pero el entrenamiento puede tardar más. Para clase, use checkpoints existentes cuando el objetivo conceptual no sea entrenar desde cero.
- **Ejecución fuera de orden.** Los notebooks están diseñados como una ruta acumulativa. Algunos conceptos y artefactos se entienden mejor si se ejecutan en orden. Si una celda indica que faltan variables, ejecute las celdas anteriores del mismo notebook desde el inicio.

8 Lista de verificación

Antes de iniciar el curso, confirme:

- La carpeta abierta es `courses/modelos_difusion_ia_generativa/`.
- El entorno de Python está activo.
- Jupyter usa el kernel del curso.
- `numpy`, `matplotlib` y `torch` importan correctamente.

- Los dos archivos `.npz` de QuickDraw existen en `data/quickdraw/processed/`.
- Los tres checkpoints `.pt` existen en `models/`.
- El notebook 00 ejecuta su primera celda de preparación sin error.
- Si se entrenará en clase, `torch.cuda.is_available()` devuelve `True`.